

ML Specification Report — playground-series-s3e5

Wine Quality — Ordinal Regression (Quadratic Weighted Kappa) • AIDE agent (20-step tree search, Claude Sonnet via shim)

All scores are taken from the run's `facts_aide.json`. Tree structure, per-step plans and per-node execution times come from `logs/2-camel-of-noble-might/journal.json`; hyperparameters from `best_solution.py` and the run's `config.yaml`; metric/target metadata from the competition `config.yaml`. Dataset shapes and the target's integer range are counted directly from the run's own `input/` directory.

Contents	
Wine Quality — Ordinal Regression (Quadratic Weighted Kappa) • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	4
Search trajectory	4
Optimization specification	6
Inference procedures	6
Evaluation & Benchmarking	7

Overview

The task is to predict an integer wine `quality` score from eleven physicochemical measurements, scored by **quadratic weighted kappa (QWK, maximize)** — an ordinal metric that penalizes a prediction by the square of how many classes it misses by. AIDE searched it over **20 steps, all 20 of which executed and scored — the only zero-bug run in this batch** — carrying a LightGBM-with-rounding draft at **0.49632** to a champion **local OOF QWK of 0.57654 at step 19, the last step of the budget**. The champion is a single seed-bagged CatBoost regressor over an interaction-augmented feature set, whose predictions are converted to classes by a multi-start Nelder-Mead threshold optimizer.

This is AIDE's best-placed episode of the batch: **public 0.62064, private 0.59073, rank 73/903 — the 92.0nd percentile**, and the score table records `local_winner = aide`. On the leaderboard it lands second of three: `lb_winner = mine` (0.59743), with AIDE at 0.59073 clearly ahead of the NVIDIA reproduce lane's 0.56974.

Why it matters. Ordinal targets with a squared-distance metric are their own problem class — sensory-quality grading, severity scoring, credit grades — and the decisive engineering choice is almost never the model but the map from a continuous score to discrete classes. This run is a clean demonstration: three quarters of AIDE's total gain came from replacing `round()` with learned cutpoints, and every subsequent model-side improvement was an order of magnitude smaller.

Data

Purpose of Data. Predict an integer wine-quality grade from eleven continuous physicochemical features, with `quality` as target and `Id` as key, scored by QWK. **Data Format** is small, clean, all-numeric **tabular CSV**. **Data Volume**, counted from the run's `input/` directory, is **2,056 training rows × 13 columns** and **1,372 test rows × 12 columns** — eleven raw predictive features (`fixed acidity`, `volatile acidity`, `citric acid`, `residual sugar`, `chlorides`, `free sulfur dioxide`, `total sulfur dioxide`, `density`, `pH`, `sulphates`, `alcohol`).

An input-purity note. The `input/` directory also contained `train_processed.csv` ($2,056 \times 23$) and `test_processed.csv` ($1,372 \times 22$), which carry ten pre-engineered chemistry features on top of the raw eleven: `free_so2_ratio`, `bound_so2`, `fixed_to_volatile_acid`, `citric_to_volatile_acid`, `total_acid`, `alcohol_x_sulphates`, `alcohol_to_density`, `alcohol_x_density`, `sugar_to_alcohol`, `acid_ph_ratio`. AIDE read the processed files from step 0 onward and never touched `train.csv`, so its feature baseline is inherited rather than discovered; its own feature contribution is the automatic interaction block described below. The effect is milder than in `s3e1` — the inherited block is ten simple ratios, not a full engineered pipeline — but it should be recorded.

Data Quality was never profiled (**no EDA artifact exists**). The one distributional fact AIDE acted on is class imbalance: it switched from `KFold` to `StratifiedKFold` at step 13 on the argument that "few very-low/very-high quality wines" make random folds unstable, and it tested inverse-frequency sample weighting at step 17 for the same reason. Measured from the run's input, the target takes six integer values, 3 through 8, and the two central classes dominate. Missing values, duplicates and train↔test drift were never measured by this run.

Annotation Guidelines. The label is an **integer grade in [3, 8]**, and the submission must contain integers — this is not a probability or ranking task. QWK's quadratic penalty means a 5-predicted-as-7 error costs four times a 5-predicted-as-6 error, which is precisely why the thresholding step dominates everything else.

Feature Set. The champion evaluates two feature sets and keeps the better:

```
23 (processed columns) - 2 (Id, quality) = 21 base features
C(6,2) = 15 top-feature pairs × 2 (product + ratio) = 30 generated features
21 + 30 = 51 features in the augmented arm
```

Group	Origin	Features
Raw physicochemical (11)	competition data, via <code>train_processed.csv</code>	<code>fixed_acidity</code> , <code>volatile_acidity</code> , <code>citric_acid</code> , <code>residual_sugar</code> , <code>chlorides</code> , <code>free_sulfur_dioxide</code> , <code>total_sulfur_dioxide</code> , <code>density</code> , <code>pH</code> , <code>sulphates</code> , <code>alcohol</code>
Inherited engineering (10)	present in <code>train_processed.csv</code>	<code>free_so2_ratio</code> , <code>bound_so2</code> , <code>fixed_to_volatile_acid</code> , <code>citric_to_volatile_acid</code> , <code>total_acid</code> , <code>alcohol_x_sulphates</code> , <code>alcohol_to_density</code> , <code>alcohol_x_density</code> , <code>sugar_to_alcohol</code> , <code>acid_ph_ratio</code>
Automatic interactions (30)	AIDE, steps 18–19	pairwise products and ratios among the six highest-importance base features, ranked by CatBoost gain from a first pass

The interaction block is generated, not designed: AIDE fits the baseline arm first, reads `get_feature_importance()`, takes the top six columns and emits every pairwise product and ratio. That is a mechanical mutation, and its measured worth is correspondingly small (see the trajectory).

Splitting strategy. `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`, stratified on the integer quality label, matching the harness's `k_fold_validation: 5`. Within each fold the model is trained three times with seeds 42, 123 and 7 and the predictions averaged, so the OOF vector is a seed-bagged continuous score. The plain `KFold` used up to step 12 was replaced at step 13 specifically to stabilize the rare extreme classes. The leaderboard was exercised: the champion's submission was scored publicly and privately.

Models & Architecture

Purpose of Architecture. Produce a continuous latent quality score whose learned cutpoints maximize QWK. **Architecture Type** is a **single-learner regression-plus-thresholding pipeline**: one CatBoost regressor trained with squared-error loss on the integer target, seed-bagged three ways within each fold, followed by an `OptimizedRounder` that learns five cutpoints by direct QWK maximization. Notably, AIDE tried ensembling repeatedly and it never worked here — the LightGBM+XGBoost blend at step 5 scored 0.4082, well below the draft, and the top-3 hyperparameter-config average at step 16 lost to the single best config. The champion is deliberately a single model.

Input Format is a dense float matrix with no categorical columns. **Input Dimension** is **51 features** in the selected (augmented) arm, 21 in the baseline arm.

Architecture Description. CatBoost regressor with `depth 4`, `l2_leaf_reg 6.0`, `learning_rate 0.02`, `bagging_temperature 0.0`, `random_strength 1.0`, `iterations 2000`, `loss_function="RMSE"`, early stopping 100 on the fold's validation split with `use_best_model=True`. These values are the output of AIDE's own randomized search at steps 11–14, not defaults.

The `OptimizedRounder` is the second half of the architecture and deserves its own description. It holds a sorted vector of five cutpoints separating the six classes, applies them with `np.digitize`, and fits them

by minimizing negative QWK under **Nelder-Mead** (`maxiter` 10000, `xatol` / `fatol` 1e-6). To escape local optima it runs **27 initializations** — a uniform half-integer start, a percentile-based start derived from the sorted prediction vector, and 25 Gaussian perturbations ($\sigma = 0.3$) of the uniform start — and keeps whichever converged cutpoint set scores highest. Multi-start was itself an AIDE discovery: step 9 introduced it and moved the metric 0.56841 $\rightarrow$ 0.57275.

Model Complexity in fits:

2 feature arms $\times$ 3 seeds $\times$ 5 folds = 30 CatBoost fits

each with a 2000-iteration ceiling at depth 4, plus 2×27 Nelder-Mead threshold optimizations. This is the smallest model in the batch by a wide margin, which is consistent with a 2,056-row training set.

Training procedures

Search trajectory

Step	Parent	What AIDE changed	OOF QWK
0	—	draft: LightGBM regression + simple rounding	0.49632
1	—	draft: LightGBM multiclass classification, argmax	0.48098
2	0	OptimizedRounder — global Nelder-Mead cutpoints	0.55672
3	2	per-fold thresholds, averaged	0.5452
4	2	multi-seed bagging of LightGBM	0.55917
5	—	draft: LightGBM + XGBoost ensemble	0.4082
6	—	draft: CatBoost regression + global rounder	0.56841
7	—	draft: CatBoost + 3-seed bagging	0.56591
8	6	repeated 3 $\times$ 5-fold CV	0.56769
9	6	multi-start threshold optimization (27 inits)	0.57275
10	9	+ seed bagging on top of multi-start	0.56897
11	9	randomized CatBoost hyperparameter search (9 configs)	0.5747
12	11	MAE loss instead of RMSE	0.56419
13	11	StratifiedKFold instead of KFold	0.57473
14	13	+ <code>bagging_temperature</code> / <code>random_strength</code> in the search	0.57589
15	14	importance-based feature pruning (bottom 25 %) — rejected internally	0.57589
16	14	top-3 config averaging — rejected internally	0.57589
17	14	inverse-frequency sample weighting — rejected internally	0.57589
18	14	+ generated pairwise interaction/ratio features	0.57596

Step	Parent	What AIDE changed	OOF QWK
19	18	+ 3-seed bagging inside each fold (champion)	0.57654

The tree opened with the configured **5 root drafts** (steps 0, 1, 5, 6, 7) and reached **depth 6**; the champion sits at depth 6 on the lineage $6 \rightarrow 9 \rightarrow 11 \rightarrow 13 \rightarrow 14 \rightarrow 18 \rightarrow 19$, i.e. it descends from the fourth of the five drafts, not the first. **10 of the 20 scored steps set a new incumbent.** Total improvement:

$$0.57654 - 0.49632 = 0.08022 \quad (16.1630 \% \text{ relative})$$

One change accounts for three quarters of the run. Step 2 replaced round-to-nearest with learned cutpoints:

$$\begin{aligned} 0.55672 - 0.49632 &= 0.0604 \\ 0.0604 / 0.08022 &= 75.29 \% \text{ of the run's total improvement} \end{aligned}$$

Everything after step 2 — a better base learner, a hyperparameter search, stratification, generated interactions, seed bagging — bought the remaining quarter between them. AIDE's own report reaches the same conclusion ("thresholding matters more than model choice"), and it is the correct reading of this competition.

The honest weakness is the plateau. Steps 15, 16 and 17 all report **exactly 0.57589**, the step-14 incumbent's score. That is not a coincidence: each of those scripts tests its new lever internally (feature pruning, top-3 config averaging, class weighting), finds it worse, falls back to the incumbent configuration and reports the incumbent's number. So **4 of 20 steps (14 through 17) returned the same value**, 20 % of the budget spent confirming that three plausible levers do nothing. AIDE also spent two of its five root drafts on approaches that were worse than the first draft — multiclass classification at 0.48098 and the two-model ensemble at 0.4082, both below the 0.49632 baseline they were meant to improve on. On the other hand, this run wasted nothing on infrastructure: **zero buggy steps, zero timeouts**, and the whole search consumed 209.5 s.

A second, subtler weakness is in the metric itself. The reported 0.57654 is computed **after fitting the five cutpoints on the same OOF prediction vector that is then scored**, so it is not an unbiased estimate — the thresholds have seen the labels they are evaluated against. AIDE tested the cleaner alternative at step 3 (fit thresholds per fold, average them), measured 0.5452, and discarded it. With only five free parameters the optimism is small, and the leaderboard bears that out (private 0.59073 is above the local figure), but the model-selection objective was nonetheless partly self-referential.

One piece of context for reading "step 0": this journal is the second leg of the run, started with `initial_journal` pointing at a 5-node journal from an earlier leg, so AIDE's memory at step 0 already contained five prior attempts on this competition.

Optimization specification

The **Loss Function** is squared error — CatBoost `loss_function="RMSE"` on the integer target treated as continuous — while **model selection is on QWK**, and the threshold layer optimizes QWK directly and non-differentiably. The two objectives are deliberately mismatched, and AIDE tested the alternative: step 12 switched the training loss to MAE and lost (0.56419 against the 0.5747 incumbent), confirming that squared error is the better latent-score objective even when the reported metric is kappa. The **Optimization Algorithm** is ordered gradient boosting for the base learner and **Nelder-Mead simplex search with 27 restarts** for the cutpoints. The **Learning Rate** is the boosting shrinkage, `learning_rate` 0.02 (the threshold optimizer has no learning rate — Nelder-Mead is derivative-free). There is **no Learning Rate Scheduler** (N/A — convergence is governed by early stopping at patience 100, not a schedule) and **no Batch Size** (N/A — full-dataset boosting).

Training Duration for the whole search was **209.5 s of executed wall-clock across 20 nodes** (mean 10.5 s, max 27.0 s) — the cheapest run in the batch by a wide margin, with no node coming within two orders of magnitude of the 1800 s harness timeout. **Training Memory Consumption Limits** are **not recorded**.

Transfer Learning is N/A (no pretrained weights; the competition forbids them); its analogue is **journal memory**, and it is visibly the driver here — step 6's plan opens by reasoning that "the previous multi-model ensemble diluted the best single-model regression signal", which is what redirects the search from LightGBM to CatBoost and onto the winning lineage. **Data Augmentation** is N/A (tabular); the analogues are the generated interaction features and **seed bagging** (seeds 42, 123, 7 averaged within each fold).

Reproducibility Standards are moderate and unverified: `StratifiedKFold(random_state=42)`, model seeds fixed at [42, 123, 7], the threshold optimizer's perturbation RNG seeded at 42. But **AIDE runs no replay or rebuild gate** — 0.57654 is what one execution printed, and the champion was never re-run to confirm it. The multi-start Nelder-Mead fit is the component most likely to move under re-execution, since 25 of its 27 starts are random perturbations.

Inference procedures

Decision Threshold is emphatically **not N/A here — it is the model's most important component**.

The CatBoost regressor emits a continuous score; the five fitted cutpoints partition it into the six classes by `np.digitize`, and the result is clipped to the observed label range [3, 8] and cast to `int`. The full inference chain for a test row is: average the fold-model predictions (test predictions are accumulated as `fold_test_preds / n_splits` across the five folds, each itself the mean of three seeds), apply the cutpoints learned on the OOF vector, clip, cast, then realign to `sample_submission.csv` id order. The submitted file therefore contains **integers, not scores**.

Inference Duration is **not recorded** separately — test scoring happens inside the fold loop. **Inference Memory Consumption Limits** are **not recorded**; the whole pipeline is a 2,000-row problem.

Evaluation & Benchmarking

Task for Performance Evaluation. Assign an integer quality grade to each of the 1,372 test wines and submit a two-column CSV in `sample_submission.csv` order. **Performance Metrics.** QWK, maximized. AIDE's local trajectory runs $0.49632 \rightarrow 0.55672 \rightarrow 0.56841 \rightarrow 0.57275 \rightarrow 0.5747 \rightarrow 0.57589 \rightarrow 0.57654$ across the draft, the threshold optimizer, the CatBoost switch, multi-start thresholds, the hyperparameter search, stratification and the final augmented seed-bagged champion. Local and leaderboard agree in direction and the local estimate is conservative:

```
0.59073 (private) - 0.57654 (local 00F) = 0.01419
0.62064 (public) - 0.59073 (private) = 0.02991
```

The 0.02991 public-to-private spread is the widest in the batch relative to the metric's scale, which is expected for a kappa computed on 1,372 test rows — QWK is a high-variance statistic when the extreme grades are rare, which is the premise AIDE itself acted on when it switched to stratified folds at step 13. The submission placed **73/903, the 92.0nd percentile**, AIDE's best placement of the five episodes.

Performance Benchmarking. All three lanes submitted to the same private leaderboard:

Lane	Approach	Private QWK	Note
My agent	from-scratch GBDT pool + tree search	0.59743	<code>lb_winner</code>
AIDE	20-step tree search, CatBoost + multi-start <code>OptimizedRounder</code>	0.59073	behind by 0.0067
NVIDIA	reproduce-agent, public-kernel replication	0.56974	behind AIDE by 0.02099

```
0.59743 - 0.59073 = 0.0067
0.59073 - 0.56974 = 0.02099
```

AIDE loses the episode by 0.0067 — a margin roughly the size of a single one of its own late-stage steps — while beating the NVIDIA reproduce lane by more than three times that. `facts_aide.json` records `local_winner = aide` and `lb_winner = mine`; the other lanes' local numbers are outside this report's source scope, so the disagreement is reported, not resolved.

The instructive part is where AIDE's remaining gap sits. It found the decisive idea (learned cutpoints) at step 2 out of 20 and then spent eighteen steps extracting 0.02 more from a single CatBoost, four of them on a flat plateau and two of its five drafts on approaches strictly worse than the baseline. It never revisited the threshold layer after step 9 — no per-class cost analysis, no ordinal-native objective, no calibration of the latent score — even though its own evidence said that layer was where the value was. A tree search with a good local move and a weak sense of which lever has remaining headroom will convert its budget into diminishing single-model tuning, which is exactly what the last six steps show.